

```
In [2]: # FEATURE SCALING + KNN MODEL TRAINING + TESTING
#IMPORT REQUIRED LIBRARIES

import pandas as pd
import numpy as np

# Train Test Split
from sklearn.model_selection import train_test_split

# Feature Scaling
from sklearn.preprocessing import StandardScaler

# KNN Model
from sklearn.neighbors import KNeighborsClassifier

# Evaluation Metrics
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

#LOAD DATASET
df = pd.read_csv("cleaned_dataset.csv")
print("DATASET LOADED SUCCESSFULLY")
print("Rows:", df.shape[0], "Columns:", df.shape[1])

#SEPARATE FEATURES AND TARGET
X = df.drop("Label", axis=1)
y = df["Label"]
print("\nFEATURES AND TARGET CREATED")
print("Feature Columns:", X.columns)

#TRAIN-TEST SPLIT
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.20,random_st
print("\nTRAIN-TEST SPLIT COMPLETED")

#FEATURE SCALING
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\nFEATURE SCALING COMPLETED")

#ELBOW METHOD
error_rates = []
for k in range(1, 21):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_scaled, y_train)
    predictions = knn.predict(X_test_scaled)
    error = np.mean(predictions != y_test)
    error_rates.append(error)

# Plot Elbow Curve
plt.figure(figsize=(10,5))
plt.plot(range(1, 21), error_rates, marker='o')
plt.title("Elbow Method For Optimal K Value")
```

```
plt.xlabel("K Value")
plt.ylabel("Error Rate")
plt.show()

print("\nChoose the K value where error becomes minimum and stable.")

#TRAIN FINAL KNN MODEL
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train_scaled, y_train)

print("\nMODEL TRAINING COMPLETED")

#TEST THE MODEL
y_pred = model.predict(X_test_scaled)
print("\nMODEL TESTING COMPLETED")

#EVALUATE PERFORMANCE
accuracy = accuracy_score(y_test, y_pred)
print("\nACCURACY")
print(accuracy * 100, "%")

#Classification report
print("\nCLASSIFICATION REPORT")
print(classification_report(y_test, y_pred))

#CONFUSION MATRIX
cm = confusion_matrix(y_test, y_pred)

print("\nCONFUSION MATRIX")
print(cm)

# Visualize Confusion Matrix
plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

#FINAL CONCLUSION

print("\nFINAL PROJECT SUMMARY")
print("1. Data was split into training and testing sets.")
print("2. Feature scaling was applied using StandardScaler.")
print("3. Elbow Method was used for selecting K.")
print("4. KNN model was trained successfully.")
print("5. Model performance was tested using accuracy and confusion matrix.")

print("\nKNN MODEL IMPLEMENTATION COMPLETED SUCCESSFULLY")
```

DATASET LOADED SUCCESSFULLY

Rows: 1594 Columns: 12

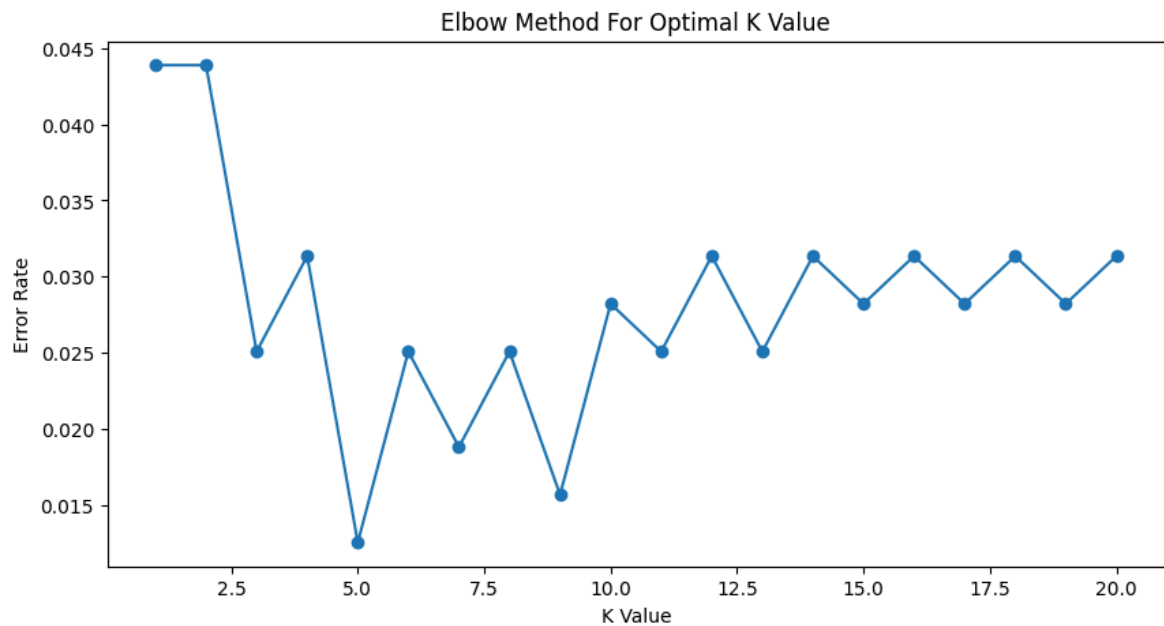
FEATURES AND TARGET CREATED

Feature Columns: Index(['Packet_Size', 'Connection_Duration', 'Error_Rate', 'Protocol',

'Source_Port', 'Destination_Port', 'Bytes_Sent', 'Bytes_Received',
'Login_Attempts', 'Failed_Logins', 'Encryption_Type'],
dtype='object')

TRAIN-TEST SPLIT COMPLETED

FEATURE SCALING COMPLETED



Choose the K value where error becomes minimum and stable.

MODEL TRAINING COMPLETED

MODEL TESTING COMPLETED

ACCURACY

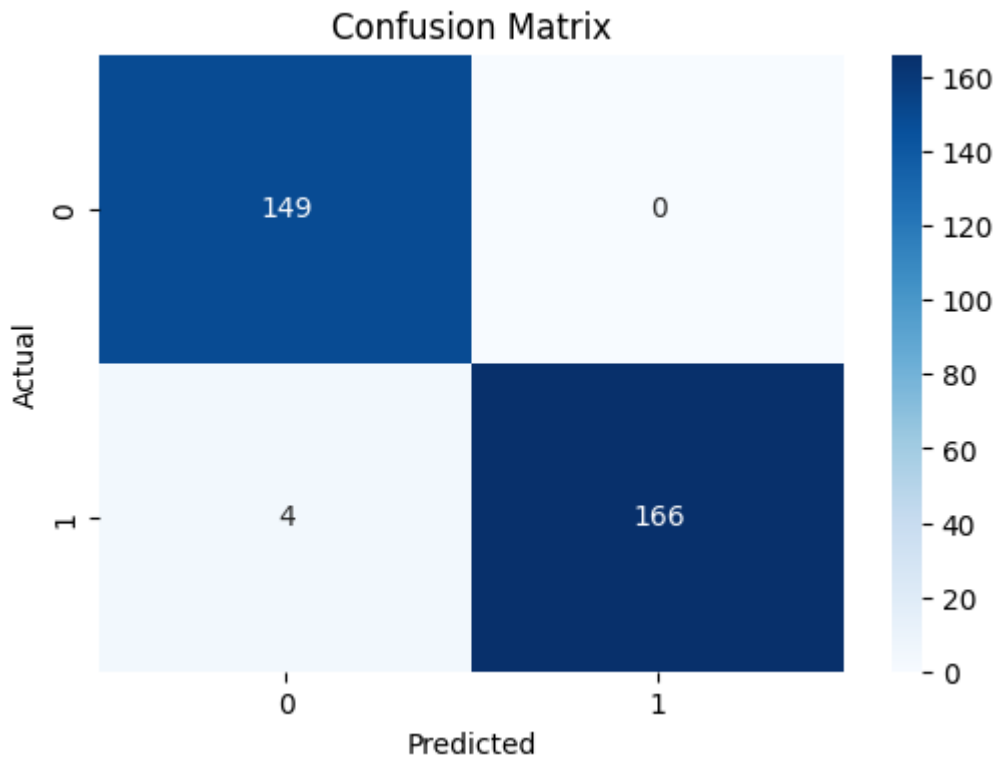
98.7460815047022 %

CLASSIFICATION REPORT

	precision	recall	f1-score	support
0	0.97	1.00	0.99	149
1	1.00	0.98	0.99	170
accuracy			0.99	319
macro avg	0.99	0.99	0.99	319
weighted avg	0.99	0.99	0.99	319

CONFUSION MATRIX

```
[[149  0]
 [  4 166]]
```



FINAL PROJECT SUMMARY

1. Data was split into training and testing sets.
2. Feature scaling was applied using StandardScaler.
3. Elbow Method was used for selecting K.
4. KNN model was trained successfully.
5. Model performance was tested using accuracy and confusion matrix.

KNN MODEL IMPLEMENTATION COMPLETED SUCCESSFULLY

```
In [4]: # APPLYING MULTIPLE MACHINE LEARNING ALGORITHMS REQUIRED
# 1. Logistic Regression
# 2. Support Vector Machine (SVM)
# 3. Random Forest
# 4. Decision Tree
# IMPORT ALGORITHMS
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier

# IMPORT EVALUATION METRICS
from sklearn.metrics import accuracy_score, classification_report, confusion_mat

# LOGISTIC REGRESSION
logistic_model = LogisticRegression(max_iter=1000)
logistic_model.fit(X_train_scaled, y_train)
logistic_pred = logistic_model.predict(X_test_scaled)

print("\nLOGISTIC REGRESSION")
print("Accuracy:", accuracy_score(y_test, logistic_pred) * 100, "%")

print("\nClassification Report:")
print(classification_report(y_test, logistic_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, logistic_pred))

# SUPPORT VECTOR MACHINE (SVM)
```

```
svm_model = SVC()
svm_model.fit(X_train_scaled, y_train)
svm_pred = svm_model.predict(X_test_scaled)
print("\nSUPPORT VECTOR MACHINE")
print("Accuracy:", accuracy_score(y_test, svm_pred) * 100, "%")

print("\nClassification Report:")
print(classification_report(y_test, svm_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, svm_pred))

# RANDOM FOREST
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train_scaled, y_train)
rf_pred = rf_model.predict(X_test_scaled)

print("\nRANDOM FOREST")
print("Accuracy:", accuracy_score(y_test, rf_pred) * 100, "%")

print("\nClassification Report:")
print(classification_report(y_test, rf_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, rf_pred))

# DECISION TREE CLASSIFIER
# IMPORT DECISION TREE
from sklearn.tree import DecisionTreeClassifier
# IMPORT METRICS
from sklearn.metrics import accuracy_score, classification_report, confusion_mat
# CREATE MODEL
dt_model = DecisionTreeClassifier(random_state=42)
# TRAIN MODEL
dt_model.fit(X_train_scaled, y_train)
# PREDICT
dt_pred = dt_model.predict(X_test_scaled)

# RESULTS
print("\nDECISION TREE")
print("Accuracy:", accuracy_score(y_test, dt_pred) * 100, "%")

print("\nClassification Report:")
print(classification_report(y_test, dt_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, dt_pred))
```

LOGISTIC REGRESSION

Accuracy: 99.05956112852664 %

Classification Report:

	precision	recall	f1-score	support
0	0.98	1.00	0.99	149
1	1.00	0.98	0.99	170
accuracy			0.99	319
macro avg	0.99	0.99	0.99	319
weighted avg	0.99	0.99	0.99	319

Confusion Matrix:

```
[[149  0]
 [ 3 167]]
```

SUPPORT VECTOR MACHINE

Accuracy: 99.37304075235109 %

Classification Report:

	precision	recall	f1-score	support
0	0.99	1.00	0.99	149
1	1.00	0.99	0.99	170
accuracy			0.99	319
macro avg	0.99	0.99	0.99	319
weighted avg	0.99	0.99	0.99	319

Confusion Matrix:

```
[[149  0]
 [ 2 168]]
```

RANDOM FOREST

Accuracy: 100.0 %

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	149
1	1.00	1.00	1.00	170
accuracy			1.00	319
macro avg	1.00	1.00	1.00	319
weighted avg	1.00	1.00	1.00	319

Confusion Matrix:

```
[[149  0]
 [ 0 170]]
```

DECISION TREE

Accuracy: 100.0 %

Classification Report:

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0	1.00	1.00	1.00	149
1	1.00	1.00	1.00	170
accuracy			1.00	319
macro avg	1.00	1.00	1.00	319
weighted avg	1.00	1.00	1.00	319

Confusion Matrix:
[[149 0]
[0 170]]

In []: